KV CACHE 显存测量 · 通俗版

显卡里
也有一个仓库

我们花了一天，想在 8 GB 显存里量清楚一件事：
你多问一个字，到底多吃掉多少地方。
以及，我一开始是怎么把它量错的。

配合「货架 / 样品入库 / 样品出库」那套仓库数据来读

2026 年 10 月 30 日 · kv-cache-lab v0.2.2

先说你那个仓库

你库里的数据长这样：三个仓——晴空仓、苍鹰仓、星空仓；十个货架，编号 HUOJIA0001 到 HUOJIA0010；货架上有样品，编号 PUTIN00002 这样。

每条入库记录都记三个数：这次入库多少、历史上累计出库多少、现在架上还剩多少。听起来很稳，对吧？

样品入库管理.csv · 其中一行

批次	商品	货架	入库	累计出库	在架库存
PUTIN00021	品牌A-洗发水	HUOJIA0001	1	24	**−23**

进了 1 件，出了 24 件，架上剩 **−23 件**。

负库存。这三个数字不可能同时是真的。

按规矩算：入库 − 累计出库 = 在架库存。1 − 24 = −23。算术一点都没错。

错的是那个 24——它属于别的批次，却被记在了这一批头上。于是账面上，这一批货"负数存在"。

这不是你填错了那么简单。它意味着：**记账的那本，和实际的货，用了两套口径，而且没人发现。**

我今天干的活，本质就是在跟同一个毛病打架——只不过战场从仓库换到了显卡里。

显卡就是一间 8 GB 的库房

你那台电脑的显卡有 8188 MiB 显存。把它当成一间仓库，里面只有三种东西。

一、常驻样品（模型本身）

就是货架上一直放着的那批货。大约 **1005 MiB**。不管你跟它说一个字还是一万字，它都占这么多，一动不动。

二、临时缓存区（KV cache）

这个才是主角。你每问一句话，模型要把"刚才说了什么"记下来，才能在后面接着答。记在哪？就记在这片临时区里。

你问得越长，这片区就越大——而且是老老实实地按长度成比例地变大。

三、过道和叉车（CUDA 上下文）

大约 **105 MiB**。不管放多少货，这个固定开销都在，挪不走。权重加上过道，实测截距是 **1110 MiB** 左右——这就是不管你怎么量化、都省不掉的那一部分。

所以「KV cache 量化」是什么？就是把缓存区里的东西**换个包装**：原来每个数字用 2 字节装（叫 f16），改成用 1 个字节出头装（叫 q8_0）。内容一样，占的地方差不多减半。

卖模型的人常说"我们把缓存减了一半"。这句话本身没错。问题是——它减的只是中间那片临时区，两头（样品和过道）一分钱不减。

那到底要量什么?

就问一句话：**每多一个 token，缓存区多吃掉多少显存?**

token 你可以理解成"字"或者"半个词"。这里按"每 1024 个 token"来算，单位是 KiB。

这个数其实能算出来

用这个模型的尺寸套公式：2（要同时记"问"和"答"）× 28 层 × 2 个注意力头 × 128 维 × 2 字节，得出 **理论上每 token 28 KiB**。

但实测总是比理论多一点

多出来的那部分，我们管它叫「过路费」。这个仓库真正想说的就是这笔过路费——因为它决定了：你以为省了一半，实际上到底省了多少。

> **打个比方：**搬家公司说"我们箱子比别家小一半，所以便宜一半"。可你要付的不只是箱子，还有车、油、司机、过路费。箱子真小了一半，但账单没少一半。

我们要做的，就是把这张账单拆开，看清楚哪一笔是箱子，哪一笔是过路费。

怎么量：四步，跟盘点一模一样

第 一 步
清场
把模型赶出显存，并确认它真走了。

第 二 步
盘点
连读五次取中位数，算库里原有占用。

第 三 步
进货
按指定长度装进来，读最高点。

第 四 步
相减
前后一减，就是这批货的成本。



右边那个双向箭头，就是"这次加载自己吃了多少"：上面那条线是进货后的高度，下面那条是进货前的。

然后换五个长度各做一遍：**2048 → 4096 → 8192 → 16384 → 32768**（每次翻一倍）。把五个差额连起来看斜率，就是"每 token 吃多少"。

第三步为什么要取最高点？因为装载的时候，仓库会短暂堆到最高。那个最高水位才是真实的占用。取平均等于把最高的那几箱货漏算了——而这正好是我一开始犯的错，第四页会讲。

为什么要用减法，不用绝对值？因为库房不是我们一家在用。桌面上开着浏览器、微信、播放器，它们占的显存时时刻刻在变。这一天里，两次测量之间桌面占用就差了 **1078 MiB**。绝对值全废，差值才能把这点抵消掉。

量出来的两个结果

结果一：每 token 的成本

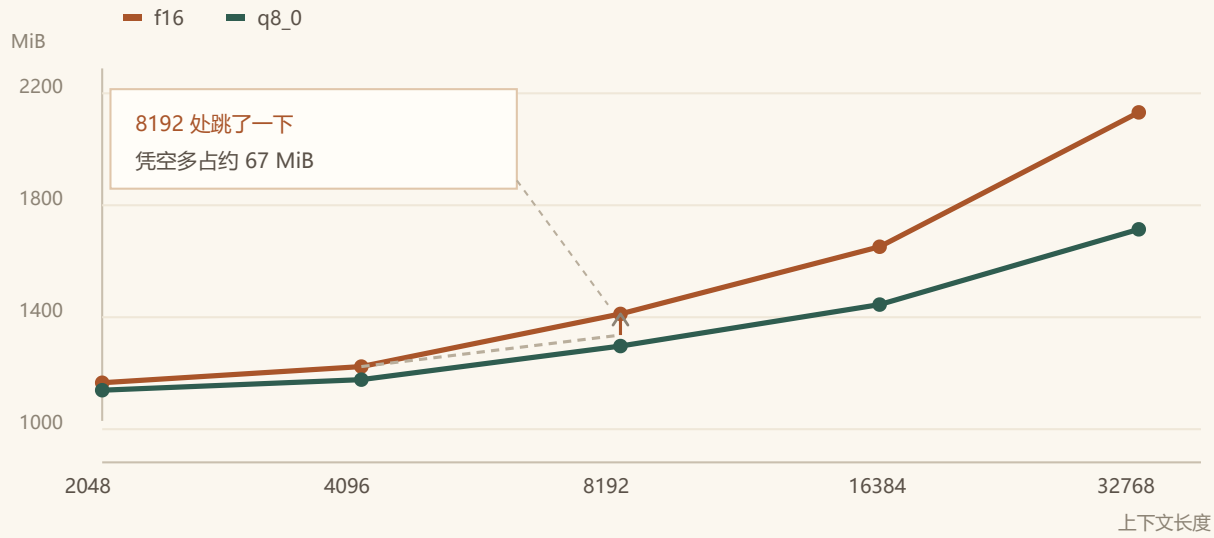
包装方式	理论该多少	实测多少	过路费
f16（每个数 2 字节）	28.00 KiB	**29.99 KiB**	+1.99
q8_0（每个数 1 字节出头）	14.88 KiB	**16.5–17.3 KiB**	+1.66…+2.43

两行都高出理论。f16 那一行很稳，误差不到 0.03；q8_0 那一行只能给区间，因为它对"基线怎么取"敏感（换成固定基线，斜率就从 17.3 掉到 16.5）。**报一位小数是假精度，所以这里给区间。**

结果二：8192 处凭空多出一块地

如果只是"每 token 固定吃一点"，五个点应该连成一条直线。但实际不是。在 **8192** 这个位置上，曲线跳了一下，凭空多占了约 **67 MiB**。

67 MiB 是什么概念？相当于你仓库里突然被划走了一块地，而且这块地跟你放多少货无关——只要超过 8192，它就一直在那儿。



那条虚线，是从 4096 那把尺子直接延伸出去的"按理该到这儿"。实际的点比它高了约 70 MiB——就是那块被凭空划走的地。两条线越往上岔得越开，因为 f16 每 token 本来就比 q8_0 贵将近一倍。

为什么这么难量：三本账

仓库里其实记着三本账，而它们从来不完全对得上。

账本	记的是什么	毛病
盘点账 nvidia-smi 读数	这一刻显卡里被占了多少	会漂。8 秒内上下晃 32 MiB；十分钟能涨 150 MiB
系统账 /api/ps 的 size_vram	模型自己说它占了多少，精确到字节	不含过道和叉车那 105 MiB
手填标签 --kv-bytes	我写的"这次用的是 f16"	**它只是我写的字，不是证据**

偏差不会凭空产生，它就长在这三本账的缝里。

最典型的一种：账实不符

我以为服务端在用 f16，就按 f16 写了标签。可实际上服务端早就被改成 q8_0 了。于是我算出来的斜率是 19.0——**它既不是 f16 的理论倍数，也不是 q8_0 的理论倍数**。

这跟你那行 PUTIN00021 一模一样：三个数字各说各话，凑在一起就成了 −23。

> 那次是怎么发现的？因为它和更早一轮 q8_0 的实测值，五个点全部吻合在 20 MiB 以内。数字太像了，反而说明我贴错了签。

我到底错在哪

四个错，都是我自己挖出来、自己修的。用仓库的话讲一遍。

> **错一：把"最高水位"记成了"平均水位"**
>
> 进货的时候，仓库会短暂堆到最高。我一开始记的是"平均堆多高"，等于把最高的那几箱货漏算了。大长度下少算了 300 到 600 MiB——**看起来就像"缓存变便宜了"**。现在改成记最高点。

> **错二：拿"昨天的库存"判断"今天的货走没走"**
>
> "模型卸载干净了吗"这件事，我原本是看显存降到某个固定数值以下。可桌面一涨，那个数就永远降不到。结果连报三次"模型可能没卸载"，而实际上模型早就走了。**只有 Ollama 自己知道货还在不在**，所以改成直接问它。

> **错三：价签是自己写的，不是系统打的**
>
> 就是上面那本"手填标签"。现在脚本会去读服务端自己的启动记录，两边对不上就**直接拒绝测量**，宁可不出数，也不出一份贴错签的数。

> **错四：我一度认定那个台阶是假的，差点把它删掉**
>
> 一次进货夹在两次盘点之间，如果这中间库房底线自己在动，相减出来的数就会跟着偏。那一轮 q8_0 的底线确实往下滑了 26 MiB，所以我第一反应是：**8192 那个台阶会不会是漂移造出来的假象？**于是换一种算法——固定基线重新拟合——台阶还在，**+53.2 MiB，位置没变**。f16 那一轮底线只动了 1 MiB，台阶照样是 67.1。所以这个台阶是真的。

这件事的教训不是"我算错了"，而是**差一点把真发现当成误差删掉**。怀疑一个结论、然后设计一个能把它杀死的检验——这一步比直接相信它更值钱，也比直接删掉它更值钱。

另外还有四个更细的，写进了代码审查记录：台阶检验在小样本下会无条件判"有台阶"、夹逼基线在缺一个点时会静默丢掉最大长度那个点、漂移探针点混进拟合、某一档读数全失败时会沿用上一档的基线。这些不影响已经发布的数字，但都记着。

最硬的一条：重测一遍

错都修完之后，我把 q8_0 那五个长度重新测了一遍。这次桌面比上次多占了 **1078 MiB**——相当于旁边凭空多开了一家店。

上下文长度	2048	4096	8192	16384	32768
第一次测	1139	1177	1297	1445	1714
重测一遍	1138	1173	1301	1441	1716
差多少	−1	−4	+4	−4	+2

五个点，全部落在 4 MiB 以内。

而库房底的绝对位置挪了 1078 MiB。也就是说：减法这一招，把一个多 GB 的环境变化，和一套测量方法的改动，一起吸收掉了，回来只差 4 MiB。

这是这一整天里最值钱的一个结果。它证明的不是模型，而是**方法能站得住**——你换个时间、换个桌面状态、换个测法，同样的五个数还会回来。

> **反过来说：**如果没有这一步，前面所有结论都只能说"在某天下午、某台机器上、某一种测法下成立"。那不是一个可以拿去用的数，那是一次运气。

所以，这一整天量出了什么

一、缓存确实按长度老实变大，但比理论贵一点点

每 token 实测比理论多约 **2 KiB**。f16 是理论 28、实测 30；q8_0 是理论 14.9、实测 17.3。**这笔过路费跟包装方式关系不大**，换了包装它还在。

二、8192 是个坎

超过 8192，会凭空多占约 **67 MiB**，而且只要超过就一直占着。这条对 8 GB 卡的用户最实际：你以为还剩很多，其实要先扣掉这一块。

三、别人的"省一半"，你要自己折算

q8_0 确实把缓存区从 28 压到 16.5–17.3，省了大约四成。但如果有人只拿缓存区那一项算给你看"省了 50%"，那是在只算箱子、不算车钱。

最后一句，说人话

> 你问得越长，显卡就被吃得越多。这件事有账可算，但账必须拿减法算，而且三本账要对齐。
>
> 我今天第二大的收获是量出了 29.99 这个数。最大的收获是：**我一度怀疑 8192 那个台阶是我自己量错造出来的，先换了种算法验一遍，它还在；后来又把整组重测一遍，它还在。**

账要对齐，减法要同时。

这两个条件少一个，你量出来的就不是显存，是当天的运气。

想自己验：仓库里有原始数据 CSV、九十多项自动检查、还有一个脚本，跑 `python tests/test_fit.py` 就能把这份报告里每个数字重算一遍，不需要显卡。